EECS 492 - Homework 2 (Winter 2026)

Due: Wednesday, February 18, 2026 at 23:59

Logistics

Submission Guidelines

You must work alone for the conceptual problems, though you may discuss ideas and high-level concepts with classmates. You may optionally work with a partner on the coding sections, though you can work alone if you want.

The due date for this assignment is **Wednesday, February 18, 2026 at 23:59**. The submission is tracked using Gradescope. The written section and programming section are graded and penalized separately. Also note that any change you make to the homework already submitted on Gradescope counts as a resubmission. If you make any changes to the assignment after the due date has passed you will be assigned a late penalty based on the number of days that have passed. For example, if you edit an assignment on February 6 and it was due on February 3 you will be assigned a 30% penalty (10% per day). This is non-negotiable.

You must submit the following files to Gradescope (note that there are two separate Gradescope submissions):

1. **A PDF file containing the solutions to the written section and the answers to the programming section questions.**. You can write your solutions by hand, use a tablet, or typeset your solutions in LaTeX. We only ask that you make it easy to read and not too verbose so that graders don't struggle to understand your writing. You must submit the written section to the Gradescope individually.
2. The completed **models.py, hw2.ipynb, cifar-10-fcn.pt and cifar-10-cnn.pt** file to the programming section. If you work with a partner, you must put both people's names on the Gradescope submission.

Late Day Tokens

To accommodate for coinciding deadlines you may have from other courses, or personal unforeseen events such as sickness, each person has 3 slip day tokens. Each token provides an automatic extension of 1 calendar day—no questions asked.

1. Each late day token covers the whole homework (both written part and programming part) for extension of 1 calendar day. For example, if you submitted written part 1 day late and programming part 2 days late. Then, if you use 1 late day token, you don't have any penalty on the written part, but will have 10% penalty on the programming part.

2. Each homework has a Gradescope submission for the late day token, in addition to the written part and programming part.
3. Late days are rounded up to the nearest integer. For example, a submission that is 4 hours late will count as one day late.
4. Extreme circumstances, like medical emergencies, etc.: In such cases, additional, no-penalty extensions will be granted. Contact your section faculty instructor with some written documentation (like doctor's note).
5. Emails requesting exceptions for other reasons will be ignored with no reply.
6. Homework turned in after three days will not be accepted.

Grading Policy

You have one week to submit a regrade request once the grades for your homework are out. We will provide solutions, along with the exact grading rubric used.

Collaboration

Collaborating with people is encouraged, but plagiarism is strictly prohibited. As mentioned in **Submission Guidelines**, you must work alone for the conceptual problems, though you may discuss ideas and high-level concepts with classmates. You may optionally work with a partner on the coding sections, though you can work alone if you want.

Generative AI

Learning how to use AI functions such as ChatGPT is important for all of us. Used properly, ChatGPT can enhance our work; used improperly, it can border on plagiarism. You are allowed to use ChatGPT or similar only for conceptual questions, and **it is strictly forbidden for programming**. If you have used ChatGPT on anything you submit for EECS 492, please include an explanation as to:

- State which model you used
- Include the prompt(s) that you gave the model
- Explain your reasoning as to why you agree (or disagree) with the response
- Mention any incorrect responses or erroneous behavior that you may have seen

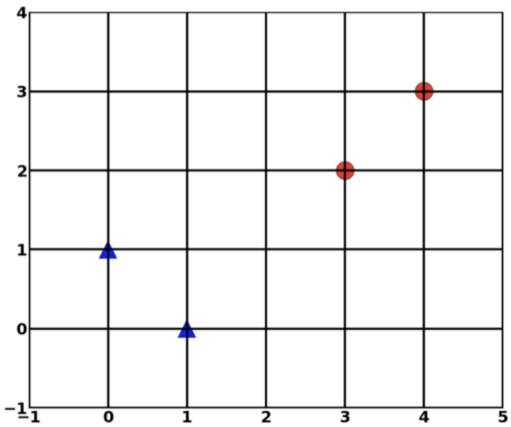
1 Written [50 Points]

1.1 Support Vector Machines[8 Points]

Assume we are given dataset $D = (x_i, y_i)_{i=1}^n$, where $x_i \in R^d$ and $y_i \in \{+1, -1\}$.

In SVM the goal is to find some hyperplane which separates the positive from the negative examples, such that the margin (the minimum distance from the decision boundary to the training points) is maximized. Let the equation for the hyperplane be $w^T x + b = 0$.

(a) You are presented with the following set of data (triangle = +1, circle = -1).



Find the hyperplane $w^T x + b = 0$ that would be used by the SVM classifier and identify the support vectors. (Draw a line to show the hyperplane and circle the support vectors. No explanation or calculations are needed.) [6 point]

(b) Will moving points which are not support vectors further away from the decision boundary affect the SVM's the w and b returned by the SVM optimization? [2 point]

1.2 Neural Networks [12 Points]

Table 1: XOR truth table

Input 1	Input 2	Output (XOR)
0	0	0
0	1	1
1	0	1
1	1	0

(a) Can a single-layer neural network with no activation functions solve the XOR problem (where you will predict the output of an XOR logic gate. The input and output are summarized in Table 1)? Can changing the same network to a multi-layer neural network with no activation functions solve the XOR problem? [2 point]
HINT: Can you draw a linear decision boundary for XOR?

(b) Consider the fully-connected neural network for binary classification in Fig. 1[1]. The network has one input layer, one hidden layer and one output layer. The hidden layer and output layer use the tanh activation function[2]:

$$\tanh(z) = \frac{e^{2z} - 1}{e^{2z} + 1}$$

The loss function is the binary cross-entropy loss:

$$\mathcal{L}(y, \hat{y}) = -\big(y \log(\hat{y}) + (1 - y) \log(1 - \hat{y})\big)$$

The derivative of the tanh function is

$$\frac{d}{dz} \tanh(z) = \frac{4}{(e^{-z} + e^{z})^2}$$

[1]This very common network architecture is often referred to as a multi-layer perceptron, or MLP.
[2]Sometimes written as $tanh(z) = \frac{e^z - e^{-z}}{e^z + e^{-z}}$.

and the derivative of the binary cross-entropy loss is

$$\frac{d}{d\hat{y}}\mathcal{L}(y, \hat{y}) = \frac{\hat{y} - y}{\hat{y}(1 - \hat{y})}$$

where y is the true label and $\hat{y}$ is the predicted output.

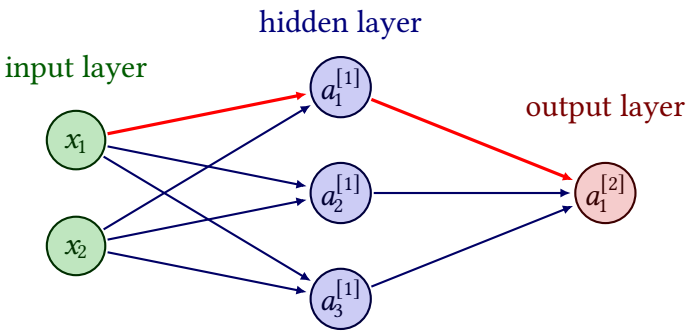


Figure 1: A fully-connected network

The expression for $\hat{y}$, the forward pass, is provided, where $\mathbf{W}^{[\ell]}$ and $\mathbf{b}^{[\ell]}$ are the weight and bias matrices for the ℓth layer, respectively:

$$\hat{y} = \tanh(\mathbf{W}^{[2]}(\tanh(\mathbf{W}^{[1]}\mathbf{x} + \mathbf{b}^{[1]})) + \mathbf{b}^{[2]})$$

By convention, we write:

- $\mathbf{W}^{[\ell]}$ = all the weights on layer ℓ,
- $\mathbf{w}_{\mathbf{i}}^{[\ell]}$ = all the weights of the ith neuron in layer ℓ, and
- $W_{i,j}^{[\ell]}$ = the weight between the ith neuron in layer ℓ and the jth neuron in layer $\ell - 1$

Using this notation, we can write the weights of the hidden layer as:

$$\mathbf{W}^{[1]} = \begin{bmatrix} W_{1,1}^{[1]} & W_{1,2}^{[1]} \\ W_{2,1}^{[1]} & W_{2,2}^{[1]} \\ W_{3,1}^{[1]} & W_{3,2}^{[1]} \end{bmatrix}$$

Suppose the network has the following specific weights and biases:

$$\mathbf{W}^{[1]} = \begin{bmatrix} 0.5 & 0.2 \\ 0.3 & 0.1 \\ 0.1 & 0.3 \end{bmatrix} \quad \mathbf{W}^{[2]} = \begin{bmatrix} 0.1 & 0.3 & 0.4 \end{bmatrix} \quad \mathbf{b}^{[1]} = \begin{bmatrix} 0.2 \\ 0.3 \\ 0.1 \end{bmatrix} \quad \mathbf{b}^{[2]} = \begin{bmatrix} 0.1 \end{bmatrix}$$

(i) Calculate the output $\hat{y}$ of a forward pass using the input [4 point]:

$$\mathbf{x} = \begin{bmatrix} 1 \\ 5 \end{bmatrix}$$

(ii) Let the neural network have two input units, three hidden units, and one output unit, as in Figure 1. The stochastic gradient descent update rules for $W_{1,1}^{[1]}$, $W_{1,1}^{[2]}$, $b_1^{[1]}$, and $b_1^{[2]}$ with learning rate α are given below, where $C^{[\ell]}$ represents the contributions from other weights in the same neuron.

$$\begin{aligned}
v &= W_{1,1}^{[1]} x_1 + b_1^{[1]} + C^{[1]} \\
a_1^{[1]} &= \tanh(v) = \frac{e^{2v} - 1}{e^{2v} + 1} \\
u &= W_{1,1}^{[2]} a_1^{[1]} + b_1^{[2]} + C^{[2]} \\
a_1^{[2]} &= \tanh(u) = \frac{e^{2u} - 1}{e^{2u} + 1}
\end{aligned}$$

The SGD update rules are:

$$\begin{aligned}
W_{1,1}^{[1]} &\leftarrow W_{1,1}^{[1]} - \alpha \left[\frac{2(a_1^{[2]} - y)}{1 - \frac{1}{e^{2u}}} W_{1,1}^{[2]} \frac{4}{(e^{-v} + e^{v})^2} x_1 \right] \\
b_1^{[1]} &\leftarrow b_1^{[1]} - \alpha \left[\frac{2(a_1^{[2]} - y)}{1 - \frac{1}{e^{2u}}} W_{1,1}^{[2]} \frac{4}{(e^{-v} + e^{v})^2} \right] \\
W_{1,1}^{[2]} &\leftarrow W_{1,1}^{[2]} - \alpha \left[\frac{2(a_1^{[2]} - y)}{1 - \frac{1}{e^{2u}}} a_1^{[1]} \right] \\
b_1^{[2]} &\leftarrow b_1^{[2]} - \alpha \left[\frac{2(a_1^{[2]} - y)}{1 - \frac{1}{e^{2u}}} \right]
\end{aligned}$$

Remember that since we are solving for a specific case, we need to account for $C^{[\ell]}$. In this case:

$$C^{[1]} = W_{1,2}^{[1]} x_2$$

$$C^{[2]} = W_{1,2}^{[2]} a_2^{[1]} + W_{1,3}^{[2]} a_3^{[1]}$$

Now consider a different input:

$$\mathbf{x} = \begin{bmatrix} 2 \\ 4 \end{bmatrix}$$

Suppose a forward pass with this input produces:

$$\mathbf{A}^{[1]} = \begin{bmatrix} 0.964 \\ 0.862 \\ 0.905 \end{bmatrix}, \quad v = 2.0, \quad u = 0.817$$

Given the true output value $y = 0.75$, use back-propagation to compute the updates for $W_{1,1}^{[1]}$, $W_{1,1}^{[2]}$, $b_1^{[1]}$, and $b_1^{[2]}$. Use stochastic gradient descent with a learning rate of $\alpha = 0.1$. [6 points]

1.3 Transformers [12 Points]

(a) Explain at an intuitive, informal level what the $\mathbf{QK}^\top$ and $(\mathbf{QK}^\top)\mathbf{V}$ terms in a self-attention layer of a transformer compute for each element of the input sequence. Assume the input is a sentence. [2 points]

(b) Suppose we have a single-headed self-attention layer with the following weight matrices for the query, key, and value:

$$\mathbf{W}_\mathbf{q} = \begin{bmatrix} 0.1 & 0.5 \\ 0.2 & 0.3 \end{bmatrix} \quad \mathbf{W}_\mathbf{k} = \begin{bmatrix} 0.3 & 0.1 \\ 0.8 & 0.3 \end{bmatrix} \quad \mathbf{W}_\mathbf{v} = \begin{bmatrix} 0.2 & 0.4 \\ 0.6 & 0.5 \end{bmatrix}$$

The transformer is given the input "I love AI!" which it encodes into a sequence of three two-dimensional vectors which are concatenated into a matrix:

$$\mathbf{X} = \begin{bmatrix} 0.6 & 0.1 \\ 0.4 & 0.5 \\ 0.1 & 0.6 \end{bmatrix}$$

(i) Calculate the query $\mathbf{Q}$, key $\mathbf{K}$, and value $\mathbf{V}$ matrices. [3 points]

(ii) Now suppose the self-attention layer produces the following query, key, and value matrices:

$$\mathbf{Q} = \begin{bmatrix} 0.12 & 0.40 \\ 0.18 & 0.30 \\ 0.15 & 0.25 \end{bmatrix} \quad \mathbf{K} = \begin{bmatrix} 0.30 & 0.10 \\ 0.50 & 0.20 \\ 0.45 & 0.15 \end{bmatrix} \quad \mathbf{V} = \begin{bmatrix} 0.20 & 0.35 \\ 0.40 & 0.45 \\ 0.35 & 0.40 \end{bmatrix}$$

Calculate the output matrix $\mathbf{Z}$ of the self-attention layer (round to four decimals). You can skip applying softmax and dividing by $\sqrt{d_k}$ for this problem. [3 points]

(c) Positional Encoding

Transformers use positional encoding to incorporate information about token position. The formulas are:

$$PE_{pos,2i} = \sin\left(\frac{pos}{10000^{2i/d}}\right) \quad \text{and} \quad PE_{pos,2i+1} = \cos\left(\frac{pos}{10000^{2i/d}}\right)$$

where pos is the position index, i determines which sine-cosine pair we're computing, and d is the embedding dimension. Use sine and cosine in radians mode.

For our input matrix $\mathbf{X}$ from part (b), we have $d = 2$ (the embedding dimension). This means we have dimensions 0 and 1 to fill:

- Dimension 0 corresponds to $2i = 0$, so $i = 0$
- Dimension 1 corresponds to $2i + 1 = 1$, so $i = 0$

(i) Calculate the positional encoding vectors PE_0, PE_1, and PE_2 for positions 0, 1, and 2 respectively. Round to four decimals. [3 points]

(ii) Add these positional encodings to the corresponding rows of $\mathbf{X}$ to create a new input matrix $\mathbf{X}'$. [1 point]

1.4 Hands-On Search [18 Points]

Consider the search tree shown in Figure 2, including nodes A through K. The state of node A is our initial state (shown in orange). Nodes F and K share the same state and are valid goals (shown in blue). The label by an edge is the cost associated with the path between the edge's vertices.

Assume that when a node is expanded, its child nodes are generated from left to right. In frontiers that use a priority queue, nodes with the same priority are popped in alphabetical order. For instance, if node B and node C have the same priority, node B is popped first, followed by node C.

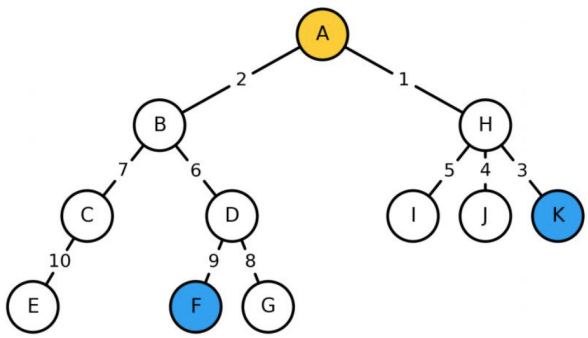


Figure 2: The Search Tree, with Path Costs

n	$h(n)$
A	4
B	3
C	8
D	4
E	15
F	0
G	14
H	1
I	7
J	2
K	0

Figure 3: Heuristic Values for the States

An admissible heuristic function is provided to you in Figure 3, which gives the value of the heuristic function for each node's state. For the following search methods, list the order in which nodes were expanded from the frontier (i.e., all `nodes` that you called `EXPAND(node)` on as seen in the pseudocode in lectures) from the start till the goal is reached. Please show your work for partial credits.

(a) Uniform Cost Search [6 pts]

(b) Greedy Best-First Search using the heuristic function provided [6 pts]

(c) A-Star Search using the heuristic function provided [6 pts]

2 Programming [50 Points]

Important: Please answer the written questions inside of this section and submit them with the written part. For submission, you will submit the written parts to HW2 Written, and the coding files/models to HW2 Coding.

2.1 Google Colab Setup

To avoid issues with Python installations and virtual environments, the code for this assignment will be completed in **Google Colab**. We recommend these steps for getting started:

(1) Download the starter code files off of Canvas.

(2) Create a folder for this HW2 programming assignment on your Google Drive and upload all of the starter code files into that folder.

(3) One of the starter code files is a notebook `.ipynb` file. To open this file, create a blank Google Colab notebook, go to 'File' > 'Open Notebook', and select the 'hw2.ipynb' starter code file. You can now run and edit this notebook to complete the assignment!

Please come to office hours if you have any issues running in Colab!

2.2 Problem Statement

In this programming section, you'll be coding neural networks to classify images. The dataset we are using is the CIFAR-10 image classification dataset:

https://www.cs.toronto.edu/~kriz/cifar.html

CIFAR-10 contains 60000 32x32 colour images in 10 classes (airplane, automobile, bird, cat, deer, dog, frog, horse, ship, truck), with 6000 images per class. There are 50000 training images and 10000 test images. You will program a fully connected neural network and a convolutional neural network as well as training/testing and experience hyperparameter search for deep learning models.

You will mainly use PyTorch (a deep learning library) for this assignment. In order to complete this assignment, you will need to learn the basics of using PyTorch. The PyTorch website has a variety of tutorials and we encourage you to explore them to get a feel for PyTorch, but there are several in particular which you should do before attempting this assignment:

1. The entire Learn the Basics tutorial. If you skip every other tutorial, at least do this one. It will cover 95% of what you need to know for this assignment.
2. We recommend you read the code to understand how to prepare Dataloaders for your models to train and test on.
3. Alternatively (or also!) you can go through the 60 Minute Blitz tutorial. This is a more in-depth starter tutorial than the Learn the Basics one, but will take a bit longer. We recommend skimming the Learn the Basics tutorial first, then going through the 60 Minute Blitz carefully.

Besides these and the dozen or so others on the PyTorch tutorial page, there are countless third-party tutorials online.

Naturally, PyTorch sports extensive documentation which you should avail yourself of whenever needed. Of particular note are:

- The `torch.nn` module docs. `torch.nn` is the cornerstone of building neural networks with PyTorch, and its docs page will constantly be helpful. It also provides a list of all the standard network layers PyTorch offers.
- The `torch.Tensor` docs. As you'll learn from the tutorials, `Tensors` (which are essentially just fancy matrices) are the core datatype of PyTorch.
- The `torchvision` library docs. Although we handled most of the stuff using `torchvision` for you, curiosity or bugs may mean you need to understand what's going on.

There are 2 files for the programming part under the homework 2 folder. The files are annotated with TODO in every location where you need to add code and have multiple comments, tips, and recommendations for how to proceed. In addition, you will never need to modify any existing code. After completion, please submit **models.py, hw2.ipynb, cifar-10-fcn.pt and cifar-10-cnn.pt** to Gradescope.

2.3 Fully Connected Neural Network, Training and Testing

Firstly, you should implement the fully connected neural network in **models.py**. There are 3 linear layers (nn.Linear) in the network. The first layer has 500 output features. The second layer has 100 output features.

Your first task is to determine the number of input features for the first layer and number of output features for the third layer. Notice that, the third layer is the classification layer. Then you need to initialize the layers. Here, we provided the second layer as an example:

```
self.linear2 = nn.Linear(500, 100)
```

After initializing layers, you should complete the forward pass using self.activation_function. Apply the activation function to the first linear layer and the second linear layer.

Finally, complete the train_and_test() function in **hw2.ipynb** which trains the model on training data as well as tests the model on testing data and returns the percentage test accuracy. For example, if the test accuracy is 0.5, return 50.

You can now train this fully connected neural network. After running, you should be able to get a test accuracy > 50%. You could also see the labels as well as predictions for 5 images in the CIFAR-10 test set.

Report your test accuracy here [5 points]:

Solution:

2.4 Convolutional Neural Network, Hyperparameter Search

Firstly, you should implement the convolutional neural network in **models.py**. This time you will additionally use nn.Conv2d, nn.MaxPool2d and nn.Flatten. The first convolutional layer (kernel size: 3x3) has 32 output channels, the second convolutional layer (kernel size: 3x3) has 64 output channels. Your first task is to determine the number of input channels for the first convolutional layer. You should only apply activation function to these 2 convolutional

layers. After 2 convolutional layer, there's a max pooling layer (kernel size: 2x2) and a flatten layer. Finally after the flatten layer, there's a linear classification layer which has 12544 input features. To see how we obtain the number 12544, you can refer to CS231n in Stanford, but it's not a requirement for this course. You can alternatively to use Tensor.size after the flatten layer to know the number of input features will be for the linear layer.

After initializing the layers, you can complete the forward pass of the convolutional neural network. It should be pretty similar to how you did it for the fully connected neural network.

Additionally, complete grid search on learning rates and activation functions in **hw2.ipynb**. **Keep the number of epochs to be 5**. **Hyperparameter Grid search will take around 30 minutes and do not close the computer while doing hyperparameter search. It might stop searching.** **Record the test accuracy in percentage (round to 1 decimal place) for each hyperparameter configuration and then complete the table below (see Table. 2). Finally, report the best test accuracy and the best configuration of hyperparameters. [5 points]**

Learning Rate \ Activation	Sigmoid	ReLU
1e-7		
1e-3		
1		

Table 2: Hyperparameter search for convolutional neural network

Solution:

Describe what you found for learning rates and explain your findings. [5 points]

Solution:

Don't forget to submit four files - models.py, hw2.ipynb, cifar-10-fcn.pt and cifar-10-cnn.pt to the Gradescope. The Gradescope tests will check whether your fully-connected neural network and convolutional neural network are correct or not. [35 points]